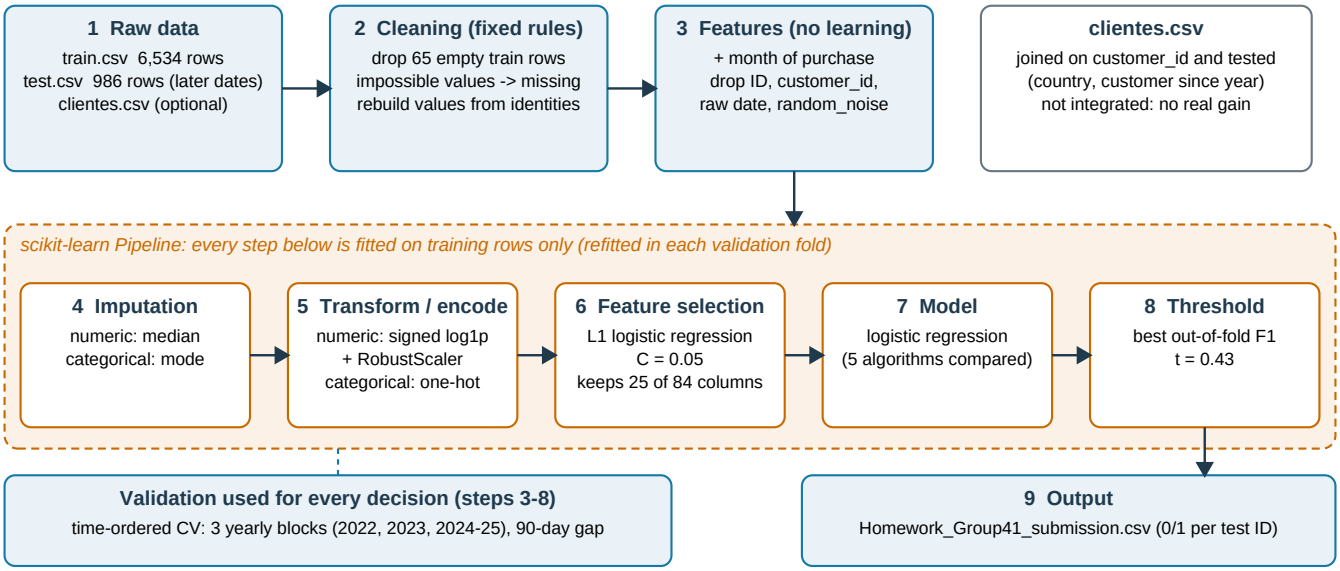
Homework: Repeat Purchase Prediction

Machine Learning 2026/27 · NOVA IMS · Group 41 · notebook *Homework_Group41.ipynb*

We predict **repeat_purchase_90d**: whether a customer of *Everything & Then Some* buys again within 90 days of a purchase. The metric is F1 on class 1. Our final pipeline is a logistic regression on the cleaned data plus the month of purchase, with L1 feature selection and a threshold tuned for F1. It reaches an F1 of **0.802** on our honest validation estimate and **0.804** on the Kaggle public leaderboard.

1. Pipeline



Steps 1-3 are deterministic and identical for train and test, so they cannot leak information. Steps 4-8 learn from data and live in one scikit-learn *Pipeline*: during validation they are refitted on the training rows of each fold, and for the submission on all train rows. *test.csv* is never used to fit anything.

2. What the data told us

- **Time matters.** Train covers 2015-07 to 2025-02, test 2025-05 to 2026-10: test is the future, 98 days after train. The share of repeat purchases moves between 0.48 and 0.63 from year to year. 94.5% of test rows are from customers seen in train.
- **The target is the customer's next purchase.** In 99.7% of train rows it equals "the same customer has another row within 90 days". So a feature may only use the past: reading a customer's later rows would reveal the answer.
- **Frequency and rhythm are the strongest signals:** purchases in the last 365 / 90 days (Spearman 0.44 / 0.39) and the usual gap between purchases (-0.42). **A 4-month cycle** exists: March, July and November have the most purchases and the lowest repeat rate.
- **Quality problems:** 65 empty train rows (the cause of all 10 duplicates), placeholder values (-4 days, amounts of 10^9), negative amounts, and missing values that are mostly structural (603 first purchases have no history).
- **Skew and redundancy:** amounts and counts are strongly right-skewed (skewness up to 27) and many columns are near-copies of each other (|rho| above 0.9), e.g. the invoice totals, or each rolling count and its spend.

3. Cleaning and the customer table

Cleaning uses **general rules**, not hand-picked rows: negative day intervals, counts or amounts become missing; values of 10^9 or more become missing; so do a mean purchase that differs from total / count, a maximum above the total spend, and a 365-day spend above the total or below the 180-day one. Where an exact identity holds (it holds in 100% of complete rows), the lost value is **rebuilt** instead of imputed: for example, mean = total / count, or days since the previous purchase = tenure for a customer's second purchase. No test row is ever removed.

clientes.csv covers every customer, but most of its columns are identifiers (company name, address), empty (customer type) or very fragmented geography. We tested the two plausible ones, country and customer-since year, as features: they added only +0.001 ROC AUC, and the second repeats what *customer_tenure_days* already says. We therefore **did not integrate** the table.

4. Pre-processing and new features

Step	Choice	Why
Missing values	median (numeric), mode (categorical), inside the pipeline	robust to skew; other plans we tried were within 0.004 ROC AUC
Transformation	signed log1p on numeric columns	limits the weight of a few huge invoices in a linear model
Scaling	RobustScaler (median / IQR)	comparable scales for logistic regression and KNN, not driven by extremes
Categoricals	one-hot; levels under 50 rows grouped as "infrequent"	80 levels in transport method, and 209 test rows have a zone never seen in train
Outliers	no row removed	large values are real large customers; the log already handles them
New feature	month of purchase (categorical)	4-month repeat cycle; best of 5 candidate groups (+0.002 ROC AUC)

The other candidate features (recency relative to the usual gap, activity trend, margin and discount share, customer table) gained 0.001 or less: they recombine information the model already has, so we left them out. Trees get the same imputation and encoding, without log and scaling, which do not change their splits.

5. Feature selection

Identifier-like columns were tested, not assumed. *ID* has no link with the target (ROC AUC 0.507) and *random_noise* none either (0.495): both are dropped. *customer_id* is an anonymised code; using it would make the model memorise customers instead of learning behaviour, so it is only a join key. The raw date is not used (test lies outside its range); only the month is derived from it. Near-constant or unrelated columns (*warehouse*, *commercial_zone*, *document_series*) are left to the selection step, which drops them.

On the remaining 84 encoded columns we compared, inside each training fold, a filter (variance, mutual information), a wrapper (RFE) and an embedded method (L1 logistic regression). Only **L1 with C = 0.05** clearly helped: **+0.011 ROC AUC on all 3 folds** with 25 of 84 columns. It keeps one or two columns per group of near-copies (e.g. tax, line discount and cost among the invoice totals), which stabilises the coefficients. Mutual information lost (-0.002) because it scores each column alone and drops the month levels, which only work together.

6. Model and assessment

Validation. Expanding time-ordered cross-validation with 3 yearly validation blocks (2022, 2023, 2024 to 2025-02). Each model is trained only on purchases made more than **90 days before** the block, because a training label looks 90 days ahead. This copies the real split. Random K-fold would mix past and future: on our data it even reverses the ranking of logistic regression and gradient boosting (0.796 vs 0.774 with time CV, 0.790 vs 0.793 with random folds).

Metrics. ROC AUC to compare models, because it does not depend on a threshold; F1 to decide, because it is the competition metric. Predicting 1 for everyone already gives F1 of about 0.75 on the last block, so that is our floor. The **honest F1** picks the threshold on blocks 1-2 and scores it on block 3, the one closest to test.

Model (with month and the pre-processing above)	ROC AUC	Honest F1	F1 minus floor
Logistic regression + L1 selection (final)	**0.796**	**0.802**	**+0.049**
Logistic regression, no selection	0.785	0.790	+0.037
KNN (k = 25)	0.776	0.780	+0.026
Gradient boosting (default)	0.774	0.777	+0.023
Decision tree (depth 6)	0.747	0.765	+0.011

Threshold. The best F1 on the out-of-fold predictions is at **0.43**, not 0.5: with 56% repeats, F1 rewards predicting 1 more readily. The F1 curve is flat between about 0.35 and 0.46, so the exact value matters little. The final model is refitted on all 6,469 clean train rows and predicts the 986 test rows with this threshold.

7. Conclusions

- The prediction rests mainly on the customer's **recent frequency** and **usual rhythm**; the calendar adds a 4-month cycle.
- A **validation that respects time** was the most important design choice: random folds would have chosen another model.
- On 6,500 rows, a **regularised linear model with L1 selection** beat more flexible models used without tuning. Our honest estimate (0.802) matched the Kaggle public score (0.804).